CSE 209 — Computer Architecture

Chapter 2 · Instructions: Language of the Computer (MIPS ISA)

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Tushar Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — MIPS ISA questions and how much they matter
Lesson 10 — Operands and the Four Design Principles — arithmetic, registers, memory, immediates, $zero, principles
Lesson 11 — Signed Numbers and Instruction Encoding — 2's complement, sign extension, R/I/J formats, encoding
Lesson 12 — Logical Operations, Decisions and Loops — shifts, masks, NOT, if/else, loops, slt, why no blt
Lesson 13 — Procedures and the Stack — register conventions, jal/jr, leaf/non-leaf, recursion, jump tables
Lesson 14 — Characters, Bytes, Strings and 32-bit Constants — lb/lh/sb/sh, strcpy, lui/ori, big constants and offsets
Lesson 15 — Branch and Jump Addressing, Far Branches — PC-relative, pseudo-direct, encoding loops, reach
Lesson 16 — The Five MIPS Addressing Modes — illustrations, calculation, examples
Lesson 17 — Synchronization and Pseudoinstructions — data race, ll/sc, atomic swap, pseudoinstructions, $at
Lesson 18 — Translating and Starting a Program; RISC vs CISC — assembler, linker, loader, DLL, JIT, fallacies, x86
Quick Notes — MIPS ISA — merged revision notes + key diagrams

> **NOTE** Slide numbers refer to Chapter_02.pdf (Tushar Sir, 69 slides, P&H 5th ed. Chapter 2). Every assembly program in these notes was run on a small MIPS simulator against the C code (script _build/verify.py); every encoding was recomputed. CISC vs RISC and x86 details go a little beyond the slides (marked textbook).

Exam Map — What the TF Papers Ask

Every Tushar-Sir-part question that this deck answers (★ = 2019-20 or later):

Session (paper)	Q	What is asked	Marks	Lesson
2023-24 (CSE 209) ★	5(a)(ii)	MIPS-v22 (opcode|rt|rd|rs|shamt|funct): encode add $t0,$t1,$t2; addi $t2,$t0,1; lw $t1,4($t0) in binary	10	L11
2023-24 (CSE 209) ★	6(b)	MIPS code for do { if ((a<<2) > b) {a += 2; b += a;} else {a += c; b += c;} } while (a < N)	13	L12
2023-24 (CSE 209) ★	6(c)	Explain PC-relative, immediate and base addressing with examples	9	L16
2023-24 (CSE 209) ★	8(c)	beq $s0,$s1,L1 is out of range — replace it by a pair of instructions with a much greater reach	5	L15
2022-23 (CSE 209) ★	11(b) NOT part	How is the NOT operation performed in MIPS? (jal-vs-j part is in the RS tracker; marks shared)		L12
2022-23 (CSE 209) ★	11(c)	Addressing modes of MIPS; illustrate the calculation for each	10	L16
2022-23 (CSE 209) ★	12(a)	How does JIT compilation improve the performance of a Java application?	5	L18
2022-23 (CSE 209) ★	12(b)	Static vs dynamic linking; advantages of dynamic linking	10	L18
2022-23 (CSE 209) ★	12(c)(i),(iii)	Build 0x20014924 in $t1; from PC = 0x1FFFF000 can one branch reach it? (12(c)(ii) is in RS tracker; 10 marks shared)	7	L15
2022-23 (CSE 209) ★	9(c)	MIPS for int fun(a,b){ if(b==0) return 0; else if(b/2==0) return fun(a*4,b/8); return fun(a+a,b/4)+a; }	10	L13
2022-23 (CSE 305) ★	5(a)	Rachel adds lh, lhu, sh, shu for multi-language support; Ross says not all are needed — agree?	7	L14
2022-23 (CSE 305) ★	5(c)	Link the object files of procedures A and B; updated addresses of the first instructions	14	L18
2022-23 (CSE 305) ★	6(b)	MIPS support for synchronization, with an example	7	L17
2022-23 (CSE 305) ★	6(c)	MIPS for if (x > y + 131075) x = 8; (x in $t1, y in $t2)	8	L14
2022-23 (CSE 305) ★	7(b)	Program uses 40 variables, MIPS has 32 registers — what does the compiler do?	7	L10
2022-23 (CSE 305) ★	7(c)	MIPS for int f(a,b,c,d){ return func(func(a,b), c+d); }	8	L13
2022-23 (CSE 305) ★	7(d)	Static vs dynamic linking; with diagrams, how DLLs are called (lazy linkage)	15	L18
2021-22 (CSE 305) ★	5(a)	Which design principle is behind each of five MIPS features (16-bit branch offset, 32 registers, …)?	15	L10
2021-22 (CSE 305) ★	6(a)	MIPS for recursive fib(n)	15	L13
2021-22 (CSE 305) ★	6(b)(ii)	PC = 0x2000 0000: can a single beq/bne set PC to 0x4000 0000? ((i) is in RS tracker)	5	L15
2021-22 (CSE 305) ★	8(c)	Addressing modes of MIPS; illustrate the calculation for each	10	L16
2020-21 (CSE 305) ★	11(c)	Efficient MIPS for testFunc(a,b,c): do { if (i>b) result += c; i++; } while (i<a)	12	L12
2020-21 (CSE 305) ★	11(d)	Illustrate how MIPS follows 'Good design demands good compromises'	5	L11
2020-21 (CSE 305) ★	12(b)	Encode L1: bne $t0,$t1,L3; L2: j LX; L3: addi $t0,$t1,1 (L1 = 1000, LX = 131072): formats and field values	10	L15
2020-21 (CSE 305) ★	13(b)	How is the address calculated in PC-relative addressing?	7	L15
2020-21 (CSE 305) ★	13(d)	Main differences between x86 and MIPS architectures	5	L18
2019-20 (CSE 305) ★	1(c)	Advantages of DLLs in C program execution; role of JIT in Java	8	L18
2019-20 (CSE 305) ★	3(a)	MIPS for: while (x>=−5) loop; if (x>y+131075) x=1; x = A[131075]+5	15	L14

Session (paper)	Q	What is asked	Marks	Lesson
2019-20 (CSE 305) ★	3(b)	How pseudoinstructions work (example); always write assembly yourself for a critical section?	12	L17
2019-20 (CSE 305) ★	3(c)	Why do MIPS arithmetic instructions demand three operands? Why only 32 registers?	8	L10
2018-19 (CSE 305)	6(a)	Addressing modes of MIPS; describe and illustrate the calculation	15	L16
2018-19 (CSE 305)	6(c)	How many MIPS instruction formats? Name and field lengths	5	L11
2018-19 (CSE 305)	7(a)	Machine code (decimal fields) for the loop loaded at 60000 (sll/add/lw/beq/j/addi/j)	10	L15
2018-19 (CSE 305)	8(a)	MIPS for recursive fun(a,b) computing a^b (b%2 test, fun(a*a,b/2)*a)	20	L13
2018-19 (CSE 305)	8(c)	beq $s0,$s1,30500 at address 005000 — what is the problem and the fix?	5	L15
2017-18 (CSE 305)	7(a)	MIPS for swap(v,k) and sort(v,n) (insertion-style nested loops)	25	L13
2017-18 (CSE 305)	7(b)	MIPS code to jump to the instruction at address 0x4000 0001	10	L15
2017-18 (CSE 305)	8(c)	How many MIPS instruction formats? Name and field lengths	10	L11
2016-17 (CSE 305)	4(c)	Compare the characteristics of typical CISC and RISC architectures (RS tracker: 'ISA (ART part)')	6	L18
2016-17 (CSE 305)	6(a)	Complete the assembly from the given machine code at 1000–1020 (sll/add/lw/bne/addi/j)	10	L15
2016-17 (CSE 305)	6(b)	MIPS support for synchronization, with an example	10	L17
2016-17 (CSE 305)	6(c)	Which design principle behind: 16-bit immediates, 3 register operands, PC-relative branches, 32 registers, fixed register fields	15	L10
2016-17 (CSE 305)	7(a)	Illustrate the five MIPS addressing modes with an example instruction each	20	L16
2015-16 (CSE 305)	5(a)	Minimal MIPS for recursive rstrcopy(char *dest, char* source)	25	L14
2015-16 (CSE 305)	5(b)	MIPS code to load 0001 0000 0011 1101 0000 1001 0000 0000_2 into $s0	6	L14
2015-16 (CSE 305)	5(c)	What are pseudoinstructions? Two examples	4	L17
2015-16 (CSE 305)	6(b)	Addressing modes of MIPS; describe and illustrate the calculation	15	L16
2015-16 (CSE 305)	6(c)	What is a data race? MIPS instructions for it and how they work	8	L17
2014-15 (CSE 305)	1(c)	Compare the characteristics of typical CISC and RISC architectures (RS tracker: 'ISA (ART part)')	8	L18
2014-15 (CSE 305)	6(a)	Complete the MIPS code of recurseSum(n) (fill the blanks)	15	L13
2014-15 (CSE 305)	6(b)	What support does MIPS provide for synchronization?	5	L17
2014-15 (CSE 305)	6(c)	Complete the decimal machine code of the loop at 6000 (sll/add/lw/beq/j/addi/j)	15	L15
2014-15 (CSE 305)	7(a)	Draw the illustrations of the five MIPS addressing modes	15	L16
2014-15 (CSE 305)	7(b)	Why no blt/bge in MIPS? Program them with MIPS instructions	10	L12
2014-15 (CSE 305)	7(c)	beq $s0,$s1,30500 at address 005000 — what is the problem and the fix?	10	L15
2013-14 (CSE 305)	6(a)	Four MIPS design principles; steps for starting a Java application	8	L10
2013-14 (CSE 305)	6(b)	MIPS code and machine code for A[3] = A[1] − A[2] (base in $s0)	12	L11
2013-14 (CSE 305)	6(c)	MIPS for int sum(int n){ if (n<=0) return 0; else return n + sum(n−1); }	12	L13
2013-14 (CSE 305)	6(d)	Branch target too far for 16 bits: how does the assembler rewrite beq $s0,$s1,L1?	3	L15

Session (paper)	Q	What is asked	Marks	Lesson
2013-14 (CSE 305)	7(a)	ll and sc; MIPS code to do A[0] = X atomically	8	L17
2013-14 (CSE 305)	7(b)	Describe MIPS addressing modes	4	L16
2012-13 (CSE 305)	1(a)	Jump address table: implement case/switch with it (example)	15	L13
2012-13 (CSE 305)	1(b)	How can you create a 32-bit constant in MIPS assembly?	5	L14
2012-13 (CSE 305)	1(c)	Loop at 80000: immediate fields of the bne and j instructions	8	L15
2012-13 (CSE 305)	1(d)	Why might the assembler have problems with 'Here: beq $s0,$s2,there … there:'? Fix	7	L15
2011-12 (CSE 305)	2(a)	MIPS design principles; one use of each in the MIPS ISA	8	L10
2011-12 (CSE 305)	2(b)	Instructions and registers used in a MIPS procedure call and their purpose	7.5	L13
2011-12 (CSE 305)	2(c)	Branch target too far for 16 bits: rewrite BEQ $s0,$s1,L1	3.5	L15
2011-12 (CSE 305)	2(d)	Describe MIPS addressing modes	10	L16
2011-12 (CSE 305)	2(e)	Three examples of the complexity of IA-32	6	L18
Related (RS tracker)	—	jal vs j (2022-23 11(b) first half), 'can j reach …' (2022-23 12(c)(ii), 2021-22 6(b)(i)) — taught in Lessons 13 and 15		L13, L15

EXAM FOCUS Chapter 2 gives the most rows of the ART part (272 marks since 2019-20). The repeating types are: (1) write MIPS for C code — loops, if/else, recursion (L12–L14, asked every year); (2) addressing modes (9 papers!) — L16 is a ready model answer; (3) encoding / decoding with branch and jump fields (L11, L15); (4) far branches and 32-bit constants (L14–L15); (5) design principles (L10); (6) linking, DLL, JIT, synchronization (L17–L18). Learn the recursion template of L13 and the L16 answer first.

Lesson 10 — Operands and the Four Design Principles

Slides 2–13 (§2.1–2.3)

1. Instruction Set and MIPS (sl. 2–3)

An instruction set is the repertoire of instructions of a computer. Different computers have different instruction sets, but with many aspects in common. MIPS (Stanford MIPS, commercialised by MIPS Technologies) is typical of modern RISC ISAs and has a large share of the embedded-core market.

2. Arithmetic Operations (sl. 4–5)

add and sub have three operands: two sources and one destination — `add a, b, c   # a gets b + c`. All arithmetic operations have this form.

```
C:     f = (g + h) - (i + j);
MIPS:  add t0, g, h      # temp t0 = g + h
       add t1, i, j      # temp t1 = i + j
       sub f, t0, t1     # f = t0 - t1
```

3. Register Operands (sl. 6–7)

- MIPS has a 32 × 32-bit register file, numbered 0–31; 32-bit data is a **word**.
- Assembler names: $t0–$t9 for temporaries, $s0–$s7 for saved variables.
- With f…j in $s0…$s4: `add $t0,$s1,$s2 / add $t1,$s3,$s4 / sub $s0,$t0,$t1`.

4. Memory Operands (sl. 8–11)

- Main memory holds composite data (arrays, structures, dynamic data). To compute on it: **load** into registers, operate, **store** back.
- Memory is **byte addressed**; words are **aligned** (address is a multiple of 4).
- MIPS is **big endian**: the most-significant byte is at the lowest address of a word (little endian: least-significant byte first).
- Registers are faster than memory; operating on memory data needs extra loads/stores. The compiler keeps variables in registers and **spills** only the less frequently used ones to memory — register optimisation is important.

```
g = h + A[8];        (g $s1, h $s2, base of A $s3)
   lw  $t0, 32($s3)      # index 8 → offset 8 × 4 = 32
   add $s1, $s2, $t0
A[12] = h + A[8];
   lw  $t0, 32($s3)
   add $t0, $s2, $t0
   sw  $t0, 48($s3)      # index 12 → offset 48
```

5. Immediate Operands and the Constant Zero (sl. 12–13)

`addi $s3, $s3, 4` adds a constant held in the instruction. There is no subtract-immediate: use a negative constant, `addi $s2, $s1, -1`. Register 0 ($zero) is the constant 0 and cannot be overwritten — useful e.g. for a move: `add $t2, $s1, $zero`.

6. The Four Design Principles ★

Principle	Where it shows in MIPS	Why

Principle	Where it shows in MIPS	Why
1. Simplicity favours regularity	All arithmetic instructions have exactly 3 register operands; all instructions are 32 bits; register fields at the same position in every format; few formats	Regularity makes the hardware simpler; simplicity gives higher performance at lower cost
2. Smaller is faster	Only 32 registers; small register file	More registers → longer wires, more decoding, slower clock; also 5-bit register fields keep instructions 32 bits
3. Make the common case fast	Immediate operands (small constants are common — no load needed); $zero; PC-relative branches (most targets are near); beq/bne only	Speed up what happens most often
4. Good design demands good compromises	Three formats (R, I, J) instead of one; 16-bit immediates and offsets; 26-bit jump field	Different formats complicate decoding but keep every instruction 32 bits — keep formats as similar as possible

Exam Approach

1. For 'which principle is behind …' name the principle and add one 'because' sentence — that sentence earns the marks.
2. Some features fit two principles (16-bit offsets: compromise + common case). Name the main one and mention the second.
3. Spilling question: keep frequently used variables in registers, spill the rest to the stack with sw/lw.

Practice — Lesson 10

(a) Actual — 2021-22 5(a). (b) Actual — 2016-17 6(c). (c) Actual — 2019-20 3(c). (d) Actual — 2022-23 (CSE 305) 7(b). (e) Actual — 2011-12 2(a) = 2013-14 6(a) first part.

(a) Which design principle is behind: (i) keeping a conditional branch's offset in 16 bits; (ii) keeping 32 registers rather than many more; (iii) a consistent format — always three registers for R-type; (iv) multiple instruction formats for flexibility; (v) a reduced instruction set where complex instructions (pseudo bgt, bge, move) are translated into basic ones? (15)

(b) Same for: (i) 16-bit immediates; (ii) always three register operands in arithmetic; (iii) PC-relative conditional branches; (iv) 32 registers; (v) register fields in the same place in each format. (15)

(c) Why does a MIPS arithmetic (e.g. multiply) instruction demand three operands? Why are there only 32 registers? (8)

(d) Chandler's program has 40 variables but MIPS has 32 registers. What does the compiler do? (7)

(e) What are the MIPS design principles? Mention one use of each in the MIPS ISA. (8)

✓ Solution

	(a)	(b)
(i)	Good design demands good compromises (a 16-bit offset keeps the instruction 32 bits; also common case — most branch targets are near)	Make the common case fast (small constants are common) — and a compromise to fit 32 bits
(ii)	Smaller is faster	Simplicity favours regularity
(iii)	Simplicity favours regularity	Make the common case fast (most branches go a short distance) — compromise
(iv)	Good design demands good compromises	Smaller is faster
(v)	Simplicity favours regularity (few simple instructions; complex ones are assembler pseudoinstructions) — also make the common case fast	Simplicity favours regularity

(c) Three operands: **simplicity favours regularity** — if every arithmetic instruction has the same form (two sources, one destination), the decoder, register-file ports and datapath are the same for all of them, which makes the hardware simpler and faster. 32 registers: **smaller is faster** — a larger register file has longer wires and more decoding, so it would need a longer clock cycle; more registers would also need wider register fields, which do not fit in a 32-bit instruction with three register operands.

(d) It performs register allocation: the most frequently used variables live in the 32 (in practice about 18 usable) registers and the rest are kept in memory (the stack frame). When one of them is needed it is loaded with lw and written back with sw — **spilling registers** (sl. 11). Because each spill costs extra instructions and slower memory accesses, the compiler spills the least frequently used variables.

(e) Table in §6, one example each (e.g. 1 — all R-type have 3 registers; 2 — 32 registers; 3 — addi; 4 — three formats with 16-bit immediates).

Lesson 11 — Signed Numbers and Instruction Encoding

Slides 14–23 (§2.4–2.5)

1. Unsigned and 2's-Complement Integers (sl. 14–17)

	n-bit range	32-bit range
Unsigned	$0 \dots 2^n - 1$	0 … 4,294,967,295
2's complement	$-2^{n-1} \dots 2^{n-1} - 1$	−2,147,483,648 … 2,147,483,647

- Bit 31 is the sign bit (1 = negative). Value = $-b_{31}\cdot 2^{31} + \Sigma\, b_i\cdot 2^i$: 1111…1100 = −2,147,483,648 + 2,147,483,644 = −4.
- Special values: 0 = 0000…0000, −1 = 1111…1111, most negative = 1000…0000 (its negation cannot be represented), most positive = 0111…1111.
- Negation: complement every bit and add 1 — +2 = 0000…0010 → 1111…1101 + 1 = 1111…1110 = −2.

2. Sign Extension (sl. 18)

To represent a number with more bits, preserve its value by replicating the sign bit to the left (unsigned values are extended with 0s). MIPS sign-extends the immediate of addi, the byte/halfword of lb/lh and the displacement of beq/bne. 8 → 16 bits: +2 = 0000 0010 → 0000 0000 0000 0010; −2 = 1111 1110 → 1111 1111 1111 1110.

3. Representing Instructions (sl. 19–23)

Instructions are encoded as 32-bit words (machine code), using a few regular formats. Register numbers: $t0–$t7 = 8–15, $t8–$t9 = 24–25, $s0–$s7 = 16–23 (also $zero 0, $at 1, $v0–$v1 2–3, $a0–$a3 4–7, $gp 28, $sp 29, $fp 30, $ra 31).

```
R-format   | op 6 | rs 5 | rt 5 | rd 5 | shamt 5 | funct 6 |   add, sub, and, or, slt, sll, jr
I-format   | op 6 | rs 5 | rt 5 |  constant or address 16  |   addi, lw, sw, beq, bne, lui
J-format   | op 6 |          address 26                     |   j, jal

rs = first source, rt = second source (R) / destination or source (I), rd = destination,
shamt = shift amount (0 if not a shift), funct = function code (extends opcode)
```

I-format constant range: $-2^{15} \dots +2^{15} - 1$. Different formats complicate decoding but allow all instructions to be 32 bits — **good design demands good compromises**; formats are kept as similar as possible (op, rs, rt always in the same place).

Worked example (sl. 21)

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

add $t0, $s1, $s2

special	$s1	$s2	$t0	0	add
0	17	18	8	0	32
000000	10001	10010	01000	00000	100000

$$00000010001100100100000000100000_2 = 02324020_{16}$$

Figure 11.1 — Encoding add $t0, $s1, $s2 (sl. 21)

```
add $t0, $s1, $s2  →  op 0 | rs $s1 = 17 | rt $s2 = 18 | rd $t0 = 8 | shamt 0 | funct 32
000000 10001 10010 01000 00000 100000  =  0000 0010 0011 0010 0100 0000 0010 0000  =
0x02324020
```

Hexadecimal is a compact way to write bit strings: 4 bits per hex digit (eca8 6420 = 1110 1100 1010 1000 0110 0100 0010 0000).

Exam Approach

1. Write the format with field widths first, then fill decimal values, then binary, then hex (group by 4).
2. Look up the operand order: add rd, rs, rt; lw rt, offset(rs); addi rt, rs, imm.
3. If the question defines its own format (MIPS-v22), follow its field order exactly — do not use standard MIPS order.

Practice — Lesson 11

(a) Actual — 2023-24 5(a)(ii). (b) Actual — 2013-14 6(b). (c) Actual — 2018-19 6(c) = 2017-18 8(c). (d) Actual — 2020-21 11(d).

(a) MIPS-v22: R-type = opcode(6) | rt(5) | rd(5) | rs(5) | shamt(5) | funct(6); I-type = opcode(6) | rt(5) | rs(5) | constant(16). $t0, $t1, $t2 = 2, 3, 4; shamt = funct = 0. Translate add $t0,$t1,$t2 (opcode 0), addi $t2,$t0,1 (opcode 6), lw $t1,4($t0) (opcode 5) to binary. (10)

(b) Write the MIPS code and machine code for A[3] = A[1] − A[2]; base of A in $s0; lw 35, sw 43, add (0, 32), sub (0, 34). (12)

(c) How many instruction formats are there in MIPS? Name each and give the length of each field. (5/10)

(d) Give an illustrative example of how MIPS follows 'Good design demands good compromises'. (5)

✓ Solution

(a) Destination of add is rd; of addi and lw it is rt (I-type has no rd).

```
add  $t0,$t1,$t2:  opcode 0 | rt $t2=4 | rd $t0=2 | rs $t1=3 | shamt 0 | funct 0
                   000000 00100 00010 00011 00000 000000          = 0x00821800
addi $t2,$t0,1:    opcode 6 | rt $t2=4 | rs $t0=2 | constant 1
                   000110 00100 00010 0000000000000001             = 0x18820001
lw   $t1,4($t0):   opcode 5 | rt $t1=3 | rs $t0=2 | constant 4
                   000101 00011 00010 0000000000000100             = 0x14620004
```

(b)

Instruction	op	rs	rt	rd / constant	shamt	funct	Binary	Hex
lw $t0, 4($s0)	35	16	8	4			100011 10000 01000 0000000000000100	8E080004
lw $t1, 8($s0)	35	16	9	8			100011 10000 01001 0000000000001000	8E090008
sub $t2, $t0, $t1	0	8	9	10	0	34	000000 01000 01001 01010 00000 100010	01095022
sw $t2, 12($s0)	43	16	10	12			101011 10000 01010 0000000000001100	AE0A000C

(c) Three formats: **R** — op 6, rs 5, rt 5, rd 5, shamt 5, funct 6; **I** — op 6, rs 5, rt 5, immediate 16; **J** — op 6, address 26. All 32 bits.

(d) MIPS wants 32-bit instructions (regularity) but lw/sw/addi need a constant and two registers. Instead of one format with a tiny constant or longer instructions, MIPS adds the I-format: the 16-bit constant reuses the rd + shamt + funct bits. Decoding becomes slightly harder (the opcode decides the format) but every instruction stays 32 bits and op, rs, rt stay in the same positions — a good compromise (sl. 23). Same for J-format's 26-bit address.

Lesson 12 — Logical Operations, Decisions and Loops

Slides 24–35 (§2.6–2.7)

1. Logical Operations (sl. 24–28)

Operation	C	Java	MIPS
Shift left	<<	<<	sll
Shift right	>>	>>>	srl
Bitwise AND	&	&	and, andi
Bitwise OR	|	|	or, ori
Bitwise NOT	~	~	nor

- Shifts use the shamt field: `sll` fills with 0s and multiplies by 2^i; `srl` fills with 0s and divides by 2^i (unsigned only).
- AND masks bits (select some bits, clear the others): $t2 = 0000 0D C0, $t1 = 0000 3C00 → and → 0000 0C00.
- OR includes bits (set some to 1): or of the same values → 0000 3DC0.
- NOT: MIPS has no not instruction; it has 3-operand NOR, and a NOR b = NOT(a OR b). So `nor $t0, $t1, $zero` gives NOT $t1 (0000 3C00 → FFFF C3FF).

EXAM-READY ANSWER 2022-23 11(b) (NOT part): MIPS performs NOT with NOR and the zero register: `nor $t0, $t1, $zero` → $t0 = ~($t1 | 0) = ~$t1. A separate NOT is unnecessary because NOR with $zero already does it — keeping the instruction set small (simplicity favours regularity).

2. Conditional Branches and If-Else (sl. 29–30)

`beq rs, rt, L1` branches if rs == rt; `bne rs, rt, L1` if rs ≠ rt; `j L1` jumps unconditionally.

```
C:  if (i == j) f = g + h; else f = g - h;      (f, g, h, i, j in $s0 … $s4)
      bne $s3, $s4, Else      # test the OPPOSITE condition and skip the then-part
      add $s0, $s1, $s2
      j   Exit
Else: sub $s0, $s1, $s2
Exit: …                       # the assembler calculates the addresses
```

3. Loops (sl. 31)

```
C:  while (save[i] == k) i += 1;     (i $s3, k $s5, base of save $s6)
Loop: sll  $t1, $s3, 2      # t1 = i × 4
      add  $t1, $t1, $s6    # t1 = &save[i]
      lw   $t0, 0($t1)      # t0 = save[i]
      bne  $t0, $s5, Exit   # exit when save[i] != k
      addi $s3, $s3, 1      # i += 1
      j    Loop
Exit: …
```

A **basic block** is a sequence of instructions with no embedded branches (except at the end) and no branch targets (except at the beginning). Compilers optimise basic blocks; advanced processors accelerate them (sl. 32).

4. slt, slti, and Why There Is No blt (sl. 33–35) ★

`slt rd, rs, rt`: rd = 1 if rs < rt else 0; `slti rt, rs, constant`. Combine with beq/bne: `slt $t0,$s1,$s2 / bne $t0,$zero,L` branches if $s1 < $s2.

EXAM-READY ANSWER Why no blt/bge (sl. 34): hardware for <, ≥ is slower than for =, ≠ (a full subtraction and sign test vs a simple comparison). Combining a comparison with a branch would put more work into one instruction and require a slower clock — **all** instructions would be penalised. beq and bne are the common case, so MIPS keeps them and builds the others from slt + beq/bne — a good design compromise.

```
blt $s1, $s2, L   →   slt $at, $s1, $s2 ;  bne $at, $zero, L      # s1 <  s2
bge $s1, $s2, L   →   slt $at, $s1, $s2 ;  beq $at, $zero, L      # s1 >= s2
bgt $s1, $s2, L   →   slt $at, $s2, $s1 ;  bne $at, $zero, L      # s1 >  s2
ble $s1, $s2, L   →   slt $at, $s2, $s1 ;  beq $at, $zero, L      # s1 <= s2
```

Signed vs unsigned (sl. 35): slt/slti compare signed, sltu/sltiu unsigned. $s0 = 1111…1111, $s1 = 0000…0001: slt gives 1 (−1 < +1); sltu gives 0 (4,294,967,295 > 1).

Exam Approach

1. Write a register map first (variable → register).
2. if: test the opposite condition and branch over the then-part; if-else needs a j Exit after the then-part.
3. while: test at the top, j back at the bottom. do-while: body first, test at the bottom and branch back if true.
4. a > b ⇔ b < a → `slt t, b, a`; a ≥ b ⇔ !(a < b) → `slt t, a, b` + beq t, $zero.
5. Shifts for × / ÷ by powers of 2; big constants need lui/ori (Lesson 14).

Practice — Lesson 12

(a) Actual — 2023-24 6(b). (b) Actual — 2020-21 11(c). (c) Actual — 2014-15 7(b).

(a) Write MIPS for: int a = 5, b = 10, c = 2; int N = 30; do { if ((a << 2) > b) { a += 2; b += a; } else { a += c; b += c; } } while (a < N); return 0; ($t0 = a, $t1 = b, $t2 = c, $t3 = N). (13)

(b) Efficient MIPS for int testFunc(int a, int b, int c) { int i = 0, result = 0; do { if (i > b) result += c; i++; } while (i < a); return result; } ($s1 = i, $s2 = result). (12)

(c) Why does MIPS support beq and bne but not blt, bge? How can you program blt, bge in MIPS? (10)

✓ Solution

(a) Register map given. (a << 2) > b ⇔ b < (a << 2).

```
main:  addi $t0, $zero, 5      # a = 5
       addi $t1, $zero, 10     # b = 10
       addi $t2, $zero, 2      # c = 2
       addi $t3, $zero, 30     # N = 30
Loop:  sll  $t4, $t0, 2        # t4 = a << 2
       slt  $t5, $t1, $t4      # t5 = 1 if b < (a<<2), i.e. (a<<2) > b
       beq  $t5, $zero, Else   # condition false -> else part
       addi $t0, $t0, 2        # a += 2
       add  $t1, $t1, $t0      # b += a
       j    Cond
Else:  add  $t0, $t0, $t2      # a += c
       add  $t1, $t1, $t2      # b += c
Cond:  slt  $t5, $t0, $t3      # t5 = 1 if a < N
       bne  $t5, $zero, Loop   # repeat while a < N
       add  $v0, $zero, $zero  # return 0
       jr   $ra
```

Simulated: the loop ends with a = 31, b = 121 — the same as the C program.

(b) a, b, c arrive in $a0, $a1, $a2; i > b ⇔ b < i.

```
testFunc: add  $s1, $zero, $zero   # i = 0
          add  $s2, $zero, $zero   # result = 0
Loop:     slt  $t0, $a1, $s1       # t0 = 1 if b < i, i.e. i > b
          beq  $t0, $zero, Skip
          add  $s2, $s2, $a2       # result += c
Skip:     addi $s1, $s1, 1         # i++
          slt  $t0, $s1, $a0       # t0 = 1 if i < a
          bne  $t0, $zero, Loop    # do ... while (i < a)
          add  $v0, $s2, $zero     # return result
          jr   $ra
```

$s1, $s2 are callee-saved: in a real function save them on the stack (or use $t registers); the question fixes $s1, $s2, so the minimal body is shown.

(c) Reason: the EXAM-READY box in §4. Programming: the four two-instruction sequences above (slt into $at, then beq/bne with $zero) — the assembler does exactly this for the pseudoinstructions blt, bge, bgt, ble.

Lesson 13 — Procedures and the Stack

Slides 36–45 (§2.8)

1. Six Steps of a Procedure Call (sl. 36)

1. Place parameters in registers ($a0–$a3).
2. Transfer control to the procedure (jal).
3. Acquire storage for the procedure (stack frame).
4. Perform the procedure's operations.
5. Place the result in a register for the caller ($v0, $v1).
6. Return to the place of call (jr $ra).

2. Register Usage (sl. 37) ★

Registers	Number	Use	Preserved across a call?
$a0–$a3	4–7	Arguments	No
$v0, $v1	2, 3	Result values	No
$t0–$t9	8–15, 24–25	Temporaries — can be overwritten by the callee	No
$s0–$s7	16–23	Saved — callee must save/restore if it uses them	Yes
$gp	28	Global pointer for static data	Yes
$sp	29	Stack pointer	Yes
$fp	30	Frame pointer	Yes
$ra	31	Return address	Yes (saved by a non-leaf callee)

3. jal and jr (sl. 38)

`jal ProcedureLabel` puts the address of the following instruction (PC + 4) in $ra and jumps to the target. `jr $ra` copies $ra to the PC — the return. jr can also be used for **computed jumps**, e.g. case/switch statements (jump-address table).

4. Leaf Procedure (sl. 39–40)

```
int leaf_example(int g, int h, int i, int j) { int f; f = (g + h) - (i + j); return f; }
leaf_example:
    addi $sp, $sp, -4     # make room for one word
    sw   $s0, 0($sp)      # save $s0 (callee-saved, used for f)
    add  $t0, $a0, $a1
    add  $t1, $a2, $a3
    sub  $s0, $t0, $t1
    add  $v0, $s0, $zero  # result
    lw   $s0, 0($sp)      # restore $s0
    addi $sp, $sp, 4
    jr   $ra              # return
```

5. Non-Leaf Procedures and Recursion (sl. 41–43)

A procedure that calls another must save on the stack its return address and any arguments/temporaries it needs after the call, and restore them afterwards.

```
int fact(int n) { if (n < 1) return 1; else return n * fact(n - 1); }
fact: addi $sp, $sp, -8     # adjust stack for 2 items
      sw   $ra, 4($sp)      # save return address
      sw   $a0, 0($sp)      # save argument
      slti $t0, $a0, 1      # test for n < 1
      beq  $t0, $zero, L1
      addi $v0, $zero, 1    # if so, result is 1
      addi $sp, $sp, 8      #   pop 2 items
      jr   $ra              #   and return
L1:   addi $a0, $a0, -1     # else decrement n
      jal  fact             # recursive call
      lw   $a0, 0($sp)      # restore original n
      lw   $ra, 4($sp)      #   and return address
      addi $sp, $sp, 8      # pop 2 items
      mul  $v0, $a0, $v0    # n * fact(n - 1)
      jr   $ra
```

6. Local Data and Memory Layout (sl. 44–45)

Local data are allocated by the callee in its **procedure frame (activation record)**: saved argument registers, saved return address, saved $s registers, local arrays and structures; $fp points to the frame's first word, $sp to the top of stack.

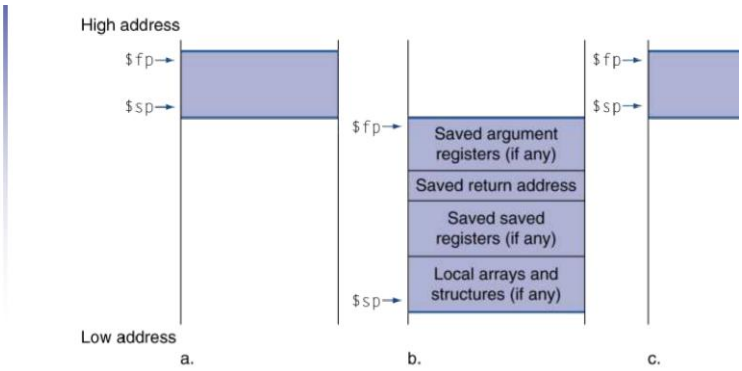


Figure 13.1 — Stack before, during and after a call (sl. 44)

```
$sp → 7fff fffc  Stack (grows down)
                 Dynamic data (heap, grows up – malloc / new)
$gp → 1000 8000  Static data (globals, constant arrays, strings) starts at 1000 0000
pc  → 0040 0000  Text (program code)
      0          Reserved
```

7. Recursion Template (use for every TF recursive function) ★

1. Handle the base case first, before building a frame (return quickly, no stack use).
2. Frame = $ra + every value needed after a call (the argument, a partial result). addi $sp,$sp,−4k; sw …
3. Set up the arguments in $a0–$a3 and jal. Values in $t registers are destroyed by the call.
4. After each call, reload what you saved, combine with $v0.
5. Restore $ra, pop the frame, jr $ra.

Exam Approach

1. State the register map ($a0 = n, result in $v0).
2. Follow the template; comment every line.
3. Two recursive calls (fib): save the first result on the stack before the second call.
4. Nested calls func(func(a,b), c+d): compute and save c + d before the first call.

Practice — Lesson 13

(a) Actual — 2022-23 9(c). (b) Actual — 2021-22 6(a). (c) Actual — 2022-23 (CSE 305) 7(c). (d) Actual — 2018-19 8(a). (e) Actual — 2014-15 6(a) / 2013-14 6(c). (f) Actual — 2017-18 7(a). (g) Actual — 2012-13 1(a). (h) Actual — 2011-12 2(b).

(a) MIPS for int fun(int a, int b) { if (b == 0) return 0; else if (b / 2 == 0) return fun(a*4, b/8); return fun(a+a, b/4) + a; } (10)

(b) MIPS for int fib(int n) { if (n == 0) return 0; else if (n == 1) return 1; else return fib(n−1) + fib(n−2); } (15)

(c) Translate int f(int a, int b, int c, int d) { return func(func(a, b), c + d); } (use lower-numbered $t registers first). (8)

(d) Minimal MIPS for the 'a^b' function: if (b == 0) return 0; else if (b % 2 == 0) return fun(a*a, b/2); return fun(a*a*, b/2) * a; (20)

(e) Complete the MIPS code of int recurseSum(int n) { if (n < 1) return 0; else return n + recurseSum(n − 1); } (10 blanks) — the same function as 2013-14's sum(n) with n <= 0. (15/12)

(f) MIPS for swap(v, k) and sort(v, n) with for (i = 0; i < n; i++) for (j = i − 1; j >= 0 && v[j] > v[j+1]; j−−) swap(v, j); (25)

(g) What is a jump address table? Implement a case/switch with it. (15)

(h) Which instructions and registers are used in a MIPS procedure call and why? (7.5)

✓ Solution

(a) a in $a0, b in $a1. b/2 is computed with sra (b ≥ 0 assumed; for negative b use div, because C truncates toward zero).

```
fun:   bne  $a1, $zero, L1     # b != 0 -> recursive cases
       add  $v0, $zero, $zero  # return 0
       jr   $ra
L1:    addi $sp, $sp, -8       # frame for $ra and a
       sw   $ra, 4($sp)
       sw   $a0, 0($sp)        # save a (needed after the 2nd call)
       sra  $t0, $a1, 1        # t0 = b / 2   (b >= 0 assumed)
       bne  $t0, $zero, L2
       sll  $a0, $a0, 2        # arg1 = a * 4
       sra  $a1, $a1, 3        # arg2 = b / 8
       jal  fun                # v0 = fun(a*4, b/8)
       j    Done
L2:    add  $a0, $a0, $a0      # arg1 = a + a
       sra  $a1, $a1, 2        # arg2 = b / 4
       jal  fun                # v0 = fun(a+a, b/4)
       lw   $t1, 0($sp)        # t1 = original a
       add  $v0, $v0, $t1      # v0 = fun(...) + a
Done:  lw   $ra, 4($sp)
       addi $sp, $sp, 8
       jr   $ra
```

(b)

```
fib:   slti $t0, $a0, 2        # n < 2 ?  (n = 0 -> 0, n = 1 -> 1)
       beq  $t0, $zero, Rec
       add  $v0, $a0, $zero    # return n
       jr   $ra
Rec:   addi $sp, $sp, -12      # room for $ra, n, fib(n-1)
       sw   $ra, 8($sp)
       sw   $a0, 4($sp)
       addi $a0, $a0, -1
       jal  fib                # v0 = fib(n-1)
       sw   $v0, 0($sp)        # keep it across the next call
       lw   $a0, 4($sp)
       addi $a0, $a0, -2
       jal  fib                # v0 = fib(n-2)
       lw   $t1, 0($sp)
       add  $v0, $v0, $t1      # fib(n-1) + fib(n-2)
       lw   $ra, 8($sp)
       addi $sp, $sp, 12
       jr   $ra
```

Simulated fib(0…12) = 0, 1, 1, …, 144 ✓. slti n < 2 covers both base cases in one test (it returns n for them).

(c) a, b, c, d arrive in $a0–$a3; func(a, b) can use them directly.

```
f:     addi $sp, $sp, -8       # frame: $ra and c+d
       sw   $ra, 4($sp)
       add  $t0, $a2, $a3      # t0 = c + d
       sw   $t0, 0($sp)        # $t0 is not preserved across a call
       jal  func               # v0 = func(a, b)   (a, b already in $a0, $a1)
       add  $a0, $v0, $zero    # arg1 = func(a, b)
       lw   $a1, 0($sp)        # arg2 = c + d
       jal  func               # v0 = func(func(a,b), c+d)
       lw   $ra, 4($sp)
       addi $sp, $sp, 8
       jr   $ra
```

(d) As printed the base case returns 0, so the function always returns 0; with `return 1` it computes a^b. The code follows the C exactly; only the base line changes.

```
fun:   bne  $a1, $zero, L1     # b != 0 -> recursive cases
       add  $v0, $zero, $zero  # return 0 (as printed; for a^b use addi $v0, $zero, 1)
       jr   $ra
L1:    addi $sp, $sp, -8
       sw   $ra, 4($sp)
       sw   $a0, 0($sp)        # save a
       andi $t0, $a1, 1        # t0 = b % 2  (b >= 0)
       mul  $a0, $a0, $a0      # arg1 = a * a
       srl  $a1, $a1, 1        # arg2 = b / 2
       bne  $t0, $zero, Odd    # decide BEFORE the call ($t0 is not preserved)
       jal  fun                # even: return fun(a*a, b/2)
       j    Done
Odd:   jal  fun                # odd: fun(a*a, b/2) * a
       lw   $t1, 0($sp)
       mul  $v0, $v0, $t1
Done:  lw   $ra, 4($sp)
       addi $sp, $sp, 8
       jr   $ra
```

TRAP $t0 (= b % 2) does not survive `jal fun`. Either branch on it before the call (as above — two call sites) or save it on the stack.

(e) Blanks in order: slti · $ra · −8 · 4($sp) · $a0 · $a0 (lw $a0, 0($sp)) · 4($sp) · addi $sp, $sp, 8 · $v0 · $ra. Complete code (2013-14 sum(n) is the same with n <= 0 ⇔ n < 1):

```
recurseSum:
       slti $t0, $a0, 1        # blank 1: slti
       beq  $t0, $zero, Recurse
       add  $v0, $zero, $zero
       jr   $ra                # blank 2: $ra
Recurse:
       addi $sp, $sp, -8       # blank 3: -8
       sw   $ra, 4($sp)        # blank 4: 4($sp)
       sw   $a0, 0($sp)        # blank 5: $a0
       addi $a0, $a0, -1
       jal  recurseSum
       lw   $a0, 0($sp)        # blank 6: $a0
       lw   $ra, 4($sp)        # blank 7: 4($sp)
       addi $sp, $sp, 8        # blank 8: the whole line
       add  $v0, $a0, $v0      # blank 9: $v0
       jr   $ra                # blank 10: $ra
```

(f) The textbook sort (§2.13). Saved registers: $s0 = i, $s1 = j, $s2 = v, $s3 = n.

```
swap:  sll  $t1, $a1, 2        # t1 = k * 4
       add  $t1, $a0, $t1      # t1 = &v[k]
       lw   $t0, 0($t1)        # temp = v[k]
       lw   $t2, 4($t1)        # t2 = v[k+1]
       sw   $t2, 0($t1)        # v[k] = v[k+1]
       sw   $t0, 4($t1)        # v[k+1] = temp
       jr   $ra
sort:  addi $sp, $sp, -20      # save $ra, $s3..$s0
       sw   $ra, 16($sp)
       sw   $s3, 12($sp)
       sw   $s2, 8($sp)
       sw   $s1, 4($sp)
       sw   $s0, 0($sp)
       move $s2, $a0           # s2 = v
       move $s3, $a1           # s3 = n
       move $s0, $zero         # i = 0
for1:  slt  $t0, $s0, $s3      # i < n ?
       beq  $t0, $zero, exit1
       addi $s1, $s0, -1       # j = i - 1
for2:  slti $t0, $s1, 0        # j < 0 ?
       bne  $t0, $zero, exit2
       sll  $t1, $s1, 2
       add  $t2, $s2, $t1      # t2 = &v[j]
       lw   $t3, 0($t2)        # v[j]
       lw   $t4, 4($t2)        # v[j+1]
       slt  $t0, $t4, $t3      # v[j] > v[j+1] ?
       beq  $t0, $zero, exit2
       move $a0, $s2           # swap(v, j)
       move $a1, $s1
       jal  swap
       addi $s1, $s1, -1       # j -= 1
       j    for2
exit2: addi $s0, $s0, 1        # i += 1
       j    for1
exit1: lw   $s0, 0($sp)        # restore and return
       lw   $s1, 4($sp)
       lw   $s2, 8($sp)
       lw   $s3, 12($sp)
       lw   $ra, 16($sp)
       addi $sp, $sp, 20
       jr   $ra
```

Simulated on [5, −2, 9, 0, 3, 3, −7, 11] → sorted; $s0–$s3 restored ✓.

(g) A **jump address table** is an array of words in memory holding the start addresses of the code for each case. The code computes k × 4, loads the table entry and does `jr` to it — one indexed jump instead of a chain of comparisons (sl. 38: jr for computed jumps). Example: switch (k) { case 0: f = i + j; break; case 1: f = g + h; break; case 2: f = g − h; break; case 3: f = i − j; break; } with f…k in $s0…$s5 and the table base in $t4:

```
sw1:   slt  $t3, $s5, $zero    # k < 0 ?
       bne  $t3, $zero, Exit
       slti $t3, $s5, 4        # k < 4 ?
       beq  $t3, $zero, Exit
       sll  $t1, $s5, 2        # t1 = 4k
       add  $t1, $t1, $t4      # t1 = &JumpTable[k]  ($t4 = table base)
       lw   $t0, 0($t1)        # t0 = JumpTable[k]
       jr   $t0                # jump to the case
L0:    add  $s0, $s3, $s4      # case 0: f = i + j
       j    Exit
L1:    add  $s0, $s1, $s2      # case 1: f = g + h
       j    Exit
L2:    sub  $s0, $s1, $s2      # case 2: f = g - h
       j    Exit
L3:    sub  $s0, $s3, $s4      # case 3: f = i - j
Exit:  jr   $ra
```

(h) Instructions: **jal** (jump and save the return address PC + 4 in $ra), **jr $ra** (return). Registers: $a0–$a3 pass arguments, $v0–$v1 return results, $ra holds the return address, $sp points to the stack where registers are saved (frames), $fp marks the frame, $s0–$s7 must be preserved by the callee, $t0–$t9 may be overwritten.

Lesson 14 — Characters, Bytes, Strings and 32-bit Constants

Slides 46–50 (§2.9–2.10)

1. Character Data (sl. 46)

- ASCII: 128 characters (95 graphic, 33 control). Latin-1: 256 (ASCII + 96 graphic).
- Unicode: 32-bit character set used in Java and C++ wide characters — most of the world's alphabets plus symbols; UTF-8 and UTF-16 are variable-length encodings.

2. Byte/Halfword Load and Store (sl. 47)

Instruction	Action
lb rt, offset(rs) / lh rt, offset(rs)	Load byte / halfword, **sign-extend** to 32 bits
lbu rt, offset(rs) / lhu rt, offset(rs)	Load byte / halfword, **zero-extend** to 32 bits
sb rt, offset(rs) / sh rt, offset(rs)	Store just the rightmost byte / halfword

EXAM-READY ANSWER 2022-23 (CSE 305) 5(a): **Agree with Ross.** Unicode text needs 16-bit loads and stores, but a store just writes the rightmost 16 bits — there is nothing to sign- or zero-extend, so 'shu' would do exactly what sh does; MIPS has only sh. For loads the extension matters: lhu (zero-extend) is what characters need, and lh (sign-extend) is useful for signed 16-bit integers. So lh, lhu and sh are enough — three instructions, not four.

3. String Copy (sl. 48–49)

```
void strcpy(char x[], char y[]) { int i = 0; while ((x[i] = y[i]) != '\0') i += 1; }   (x $a0,
y $a1, i $s0)
strcpy: addi $sp, $sp, -4      # save $s0
        sw   $s0, 0($sp)
        add  $s0, $zero, $zero # i = 0
L1:     add  $t1, $s0, $a1     # address of y[i]
        lbu  $t2, 0($t1)       # t2 = y[i]
        add  $t3, $s0, $a0     # address of x[i]
        sb   $t2, 0($t3)       # x[i] = y[i]
        beq  $t2, $zero, L2    # exit if y[i] == 0
        addi $s0, $s0, 1       # i++
        j    L1
L2:     lw   $s0, 0($sp)       # restore $s0
        addi $sp, $sp, 4
        jr   $ra
```

Characters are bytes, so the index is not multiplied by 4.

4. 32-bit Constants (sl. 50) ★

Most constants fit in 16 bits. For the occasional 32-bit constant: `lui rt, constant` copies the 16-bit constant to the left 16 bits of rt and clears the right 16 bits; then `ori` fills the right half.

```
lui $s0, 61          # $s0 = 0000 0000 0111 1101 | 0000 0000 0000 0000   (61 = 0x003D)
ori $s0, $s0, 2304   # $s0 = 0000 0000 0111 1101 | 0000 1001 0000 0000   (2304 = 0x0900) =
0x003D0900
```

TRAP Use ori, not addi, for the low half: addi sign-extends, so a low half ≥ 0x8000 would subtract from the upper half. Offsets bigger than 16 bits (A[131075] → 524300 bytes) also need lui: build the offset in a register and add the base.

Exam Approach

1. Split the constant in hex: upper 16 bits → lui, lower 16 bits → ori.
2. 131075 = 0x0002 0003 → lui 2, ori 3.
3. Strings: lbu/sb, index not ×4, stop at '\0'.
4. Tail recursion can be a plain jump (no frame needed).

Practice — Lesson 14

(a) Actual — 2019-20 3(a). (b) Actual — 2022-23 (CSE 305) 6(c). (c) Actual — 2015-16 5(b) / 2012-13 1(b). (d) Actual — 2015-16 5(a).

(a) MIPS for: (i) sum = 0; x = 100; while (x >= −5) { sum = sum + x; x = x − 1; } (ii) if (x > y + 131075) x = 1; (iii) x = A[131075] + 5; (15)

(b) MIPS for if (x > y + 131075) x = 8; with x in $t1, y in $t2. (8)

(c) MIPS to load 0001 0000 0011 1101 0000 1001 0000 0000_2 into $s0. How can you create any 32-bit constant? (6/5)

(d) Minimal MIPS for void rstrcopy(char *dest, char* source) { if (*source) {* dest++ = *source++; rstrcopy(dest, source); } else* dest = '\0'; } (dest $a0, source $a1). (25)

✓ Solution

(a) Map: sum $s0, x $s1 (part i); x $s0, y $s1 (part ii); x $s0, base of A $s2 (part iii).

```
frag1: add  $s0, $zero, $zero  # sum = 0
       addi $s1, $zero, 100    # x = 100
Loop:  slti $t0, $s1, -5       # t0 = 1 if x < -5
       bne  $t0, $zero, Exit   # leave when !(x >= -5)
       add  $s0, $s0, $s1      # sum = sum + x
       addi $s1, $s1, -1       # x = x - 1
       j    Loop
Exit:  jr   $ra
```

```
frag2: lui  $t0, 2             # t0 = 0x0002_0000 = 131072
       ori  $t0, $t0, 3        # t0 = 131075 (too big for a 16-bit immediate)
       add  $t0, $s1, $t0      # t0 = y + 131075
       slt  $t1, $t0, $s0      # t1 = 1 if (y + 131075) < x
       beq  $t1, $zero, Skip
       addi $s0, $zero, 1      # x = 1
Skip:  jr   $ra
```

```
frag3: lui  $t0, 8             # t0 = 0x0008_0000
       ori  $t0, $t0, 12       # t0 = 0x0008_000C = 131075 x 4 = 524300
       add  $t0, $t0, $s2      # t0 = &A[131075]
       lw   $t1, 0($t0)        # t1 = A[131075]
       addi $s0, $t1, 5        # x = A[131075] + 5
       jr   $ra
```

(i) simulated sum = 5035 = $\Sigma_{x=-5}^{100}$ x. (iii) 131075 × 4 = 524,300 = 0x0008 000C does not fit the 16-bit offset, so the address is built with lui/ori first.

(b)

```
frag:  lui  $t0, 2            # t0 = 131072
       ori  $t0, $t0, 3       # t0 = 131075
       add  $t0, $t2, $t0     # t0 = y + 131075
       slt  $t3, $t0, $t1     # t3 = 1 if y + 131075 < x
       beq  $t3, $zero, Skip
       addi $t1, $zero, 8     # x = 8
Skip:  jr   $ra
```

(c) 0001 0000 0011 1101 | 0000 1001 0000 0000 = 0x103D 0900: `lui $s0, 0x103D` (4157) then `ori $s0, $s0, 0x0900` (2304). In general: lui with the upper 16 bits, ori with the lower 16 bits (or the pseudoinstruction li, which the assembler expands to the same pair).

(d) Faithful recursive version (saves $ra around the call):

```
rstrcopy: lbu  $t0, 0($a1)     # t0 = *source
          beq  $t0, $zero, End # *source == '\0' ?
          sb   $t0, 0($a0)     # *dest = *source
          addi $a0, $a0, 1     # dest++
          addi $a1, $a1, 1     # source++
          addi $sp, $sp, -4
          sw   $ra, 0($sp)
          jal  rstrcopy        # rstrcopy(dest, source)
          lw   $ra, 0($sp)
          addi $sp, $sp, 4
          jr   $ra
End:      sb   $zero, 0($a0)   # *dest = '\0'
          jr   $ra
```

Minimal version: the recursive call is the last action (tail recursion), so it can be a jump and needs no stack at all — 7 instructions:

```
rcopy: lbu  $t0, 0($a1)        # t0 = *source
       sb   $t0, 0($a0)        # copy it (the final '\0' too)
       beq  $t0, $zero, Done
       addi $a0, $a0, 1        # dest++
       addi $a1, $a1, 1        # source++
       j    rcopy              # tail call = jump, no stack frame
Done:  jr   $ra
```

Both versions were simulated on "BUET CSE" — the copy, including the terminating '\0', is correct.

Lesson 15 — Branch and Jump Addressing, Far Branches

Slides 51–54 (§2.10)

1. Branch Addressing — PC-Relative (sl. 51) ★

Branches give opcode, two registers and a 16-bit target field. Most branch targets are near (forward or backward), so the field is a **word offset relative to PC + 4**:

Target = (PC + 4) + offset × 4 **offset = (Target − (PC + 4)) / 4**
Reach: offset ∈ [−32768, +32767] words → about ±128 KB around PC + 4

2. Jump Addressing — Pseudo-Direct (sl. 52)

j and jal targets could be anywhere in the text segment, so the instruction holds a 26-bit word address:

Target = (PC + 4)[31:28] : (address × 4) **address field = Target[27:2]**
Reach: any word inside the same 256 MB block as PC + 4

3. Target Addressing Example (sl. 53)

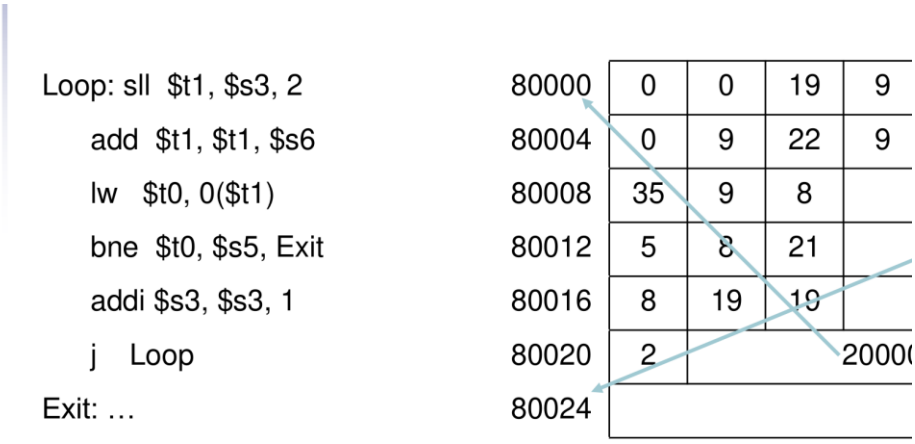


Figure 15.1 — The while loop at 80000 in decimal machine code (sl. 53)

bne at 80012 jumps to Exit = 80024: offset = (80024 − 80016)/4 = **2**. j at 80020 jumps to Loop = 80000: field = 80000/4 = **20000**.

4. Branching Far Away (sl. 54) ★

If the branch target is too far to encode in 16 bits, the assembler rewrites the code with the inverted condition and a jump:

```
beq $s0, $s1, L1        →       bne $s0, $s1, L2      # skip the jump if NOT equal
                                j   L1                # 26-bit field: far target
                         L2:    …
```

MEMORIZE Reachability checks: branch — is (Target − PC − 4)/4 an integer in [−32768, 32767]? jump — are the top 4 bits of Target equal to those of PC + 4, and is Target a multiple of 4? Anything else needs lui/ori + jr (any 32-bit, word-aligned address).

Exam Approach

1. Number every instruction address first (start + 4k).
2. Branch field = (target − (address + 4))/4 — may be negative.
3. Jump field = target / 4 (when the top 4 bits are 0) — in decimal questions just divide by 4.
4. Far branch: invert the condition (beq ↔ bne) to skip over a j.

Practice — Lesson 15

(a) Actual — 2018-19 7(a) (same code as 2014-15 6(c), 2012-13 1(c)). (b) Actual — 2016-17 6(a). (c) Actual — 2020-21 12(b). (d) Actual — 2022-23 12(c)(i),(iii); 2021-22 6(b)(ii); 2017-18 7(b). (e) Actual — 2023-24 8(c); 2018-19 8(c) = 2014-15 7(c); 2013-14 6(d), 2012-13 1(d), 2011-12 2(c). (f) Actual — 2020-21 13(b).

(a) Loop: sll $t1,$s2,2; add $t1,$t1,$s6; lw $t0,0($t1); beq $t0,$s5,Lp1; j Exit; Lp1: addi $s2,$s2,1; j Loop; Exit: — loaded at 60000. Write the machine code in decimal. (10)

(b) Recover the assembly from the machine code at 1000–1020 (2016-17 6(a) table). (10)

(c) L1: bne $t0,$t1,L3; L2: j LX; L3: addi $t0,$t1,1. Opcodes j 2, bne 5, addi 8; $t0 = 8, $t1 = 9; L1 = 1000, L2 = 1004, L3 = 1008, LX = 131072. Give the format and field values. (10)

(d) (i) Put 0010 0000 0000 0001 0100 1001 0010 0100_2 in $t1. (iii) PC = 0x1FFF F000: can a single branch reach that address? (ii′) PC = 0x2000 0000: can a single beq/bne reach 0x4000 0000? (ii″) Write MIPS to jump to 0x4000 0001.

(e) beq $s0,$s1,L1 is out of range — replace it. What is the problem with 'beq $s0, $s1, 30500' stored at address 005000?

(f) How is the address calculated in PC-relative addressing? (7)

✓ Solution

(a) Exit is at 60028.

Address	Instruction	op	rs	rt	rd	shamt	funct / imm / addr
60000	sll $t1, $s2, 2	0	0	18	9	2	0
60004	add $t1, $t1, $s6	0	9	22	9	0	32
60008	lw $t0, 0($t1)	35	9	8			0
60012	beq $t0, $s5, Lp1	4	8	21			1 = (60020 − 60016)/4
60016	j Exit	2					15007 = 60028/4
60020	addi $s2, $s2, 1	8	18	18			1
60024	j Loop	2					15000 = 60000/4

2014-15 6(c) is the same code at 6000: blanks are shamt 2; 9, 22, 9; 9, 8; beq offset 1; j Exit = 6028/4 = **1507**; j Loop = 6000/4 = **1500**. 2012-13 1(c) is slide 53: bne immediate **2**, j field **20000**.

(b) Decode the fields (register numbers: 8 $t0, 15 $t7, 16 $s0, 17 $s1, 23 $s7):

```
1000:  0 | 00000 | 10000 | 01111 | 00010 | 000000  →  sll  $t7, $s0, 2
1004:  0 | 01111 | 10001 | 01111 | 00000 | 100000  →  add  $t7, $t7, $s1
1008: 35 | 01111 | 01000 | 0                       →  lw   $t0, 0($t7)
1012:  5 | 01000 | 10111 | 1                       →  bne  $t0, $s7, Exit   (target 1016 + 1×4
= 1020)
1016:  8 | 10000 | 10000 | 1                       →  addi $s0, $s0, 1
1020:  2 | 250                                     →  j    Loop             (250 × 4 = 1000)
```

Note: as printed the bne offset is 1, which points at 1020; for an Exit after the j (1024) the offset would be 2 — say so in the exam and keep the printed value.

(c)

```
1000  L1: bne $t0, $t1, L3   I-format | 5 | 8 | 9 | 1          offset = (1008 − 1004)/4 = 1
1004  L2: j LX              J-format | 2 | 32768              131072 / 4 = 32768
1008  L3: addi $t0, $t1, 1  I-format | 8 | 9 | 8 | 1          rs = $t1 = 9, rt = $t0 = 8
```

(d)

```
(i)   0x2001 4924:  lui $t1, 0x2001          # 8193
                    ori $t1, $t1, 0x4924     # 18724
(iii) PC + 4 = 0x1FFF F004; offset = (0x2001 4924 − 0x1FFF F004)/4 = 0x5648 = 22088 ≤ 32767
      → YES: e.g. beq $zero, $zero, 22088 reaches it (target is word aligned).
(ii') 0x4000 0000 − 0x2000 0004 = 0x1FFF FFFC bytes ≈ 134 M words ≫ 32767 → NO single branch.
      (j also fails: top 4 bits 0010 ≠ 0100.)  Use: lui $t0, 0x4000 ; jr $t0
(ii") lui $t0, 0x4000 ; ori $t0, $t0, 0x0001 ; jr $t0
```

TRAP 0x4000 0001 is not word aligned. MIPS instructions start at multiples of 4, so no j/branch can encode it, and jr to it raises an address-error exception. Write the lui/ori/jr sequence (what the question asks) and state this problem; if 0x4000 0000 was meant, lui + jr is enough.

(e) Out of range: `bne $s0, $s1, L2 / j L1 / L2:` (§4) — 2023-24 8(c), 2013-14 6(d), 2011-12 2(c). 'beq $s0, $s1, 30500' at 005000: read the addresses as hex (as written with leading zeros) — distance 0x30500 − 0x5004 = 0x2B4FC bytes = 44,351 words > 32,767, so the 16-bit offset cannot hold it. Fix: `bne $s0, $s1, Next ; j 0x30500 ; Next:` (j field 0x30500/4 = 0xC140, same 256 MB block). (Read as decimal the offset would be (30500 − 5004)/4 = 6374, which fits — then mention that an assembly branch normally names a label; the assembler, not the programmer, computes the offset.) 2012-13 1(d) 'Here: beq $s0,$s2,there … there:' — when 'there' is more than 2^{15} words away the assembler cannot encode it; it rewrites with bne + j.

(f) Target = PC + 4 + sign-extended 16-bit offset × 4. The PC has already been incremented by 4 when the branch executes; the offset counts words, so it is shifted left 2; the ALU (branch adder) adds it. Example: beq at 80012 with offset 2 → 80016 + 8 = 80024.

Lesson 16 — The Five MIPS Addressing Modes

Slide 55 (§2.10) — asked in 9 papers

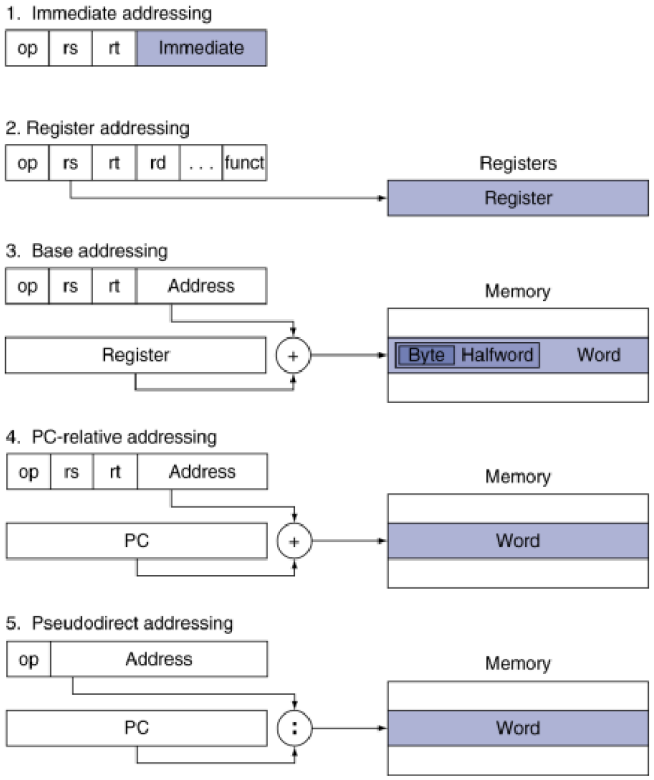


Figure 16.1 — MIPS addressing mode summary (sl. 55)

Mode	Operand is …	Calculation	Example
1. Immediate	a constant inside the instruction	operand = sign-extend(imm16)	addi $s3, $s3, 4
2. Register	a register	operand = R[rs], R[rt]	add $t0, $s1, $s2
3. Base (displacement)	in memory at register + constant	address = R[rs] + sign-extend(imm16)	lw $t0, 32($s3)
4. PC-relative	a branch target	target = PC + 4 + sign-extend(imm16) × 4	beq $s0, $s1, L1
5. Pseudo-direct	a jump target	target = (PC + 4)[31:28] : addr26 : 00	j Loop

```
1. Immediate    | op | rs | rt |   Immediate   |
2. Register     | op | rs | rt | rd | … | funct |  → Registers: R[rs], R[rt]
3. Base         | op | rs | rt |   Address     |  → R[rs] ──(+)── memory byte/half/word
4. PC-relative  | op | rs | rt |   Address ×4  |  → PC+4 ──(+)── memory word (instruction)
5. Pseudodirect | op |        Address ×4         |  → PC+4[31:28] ──( : )── memory word
```

EXAM-READY ANSWER Model answer (all addressing-mode questions): name the five modes, draw the field box for each, write the calculation and one example — exactly the table and sketch above. For 'illustrate the calculation process' add a numeric example: lw $t0, 32($s3) with $s3 = 1000 → address 1032; beq at 80012 with offset 2 → 80024; j 20000 at 80020 → (0000) : 20000 × 4 = 80000.

Practice — Lesson 16

Actual — 2023-24 6(c); 2022-23 11(c); 2021-22 8(c); 2018-19 6(a); 2016-17 7(a); 2015-16 6(b); 2014-15 7(a); 2013-14 7(b); 2011-12 2(d).

(a) Explain PC-relative, immediate and base addressing modes with examples. (9)

(b) What are the different addressing modes of MIPS? Illustrate the calculation process for each. (10–15)

✓ Solution

(a) PC-relative: the 16-bit field of beq/bne is a word offset from PC + 4 — target = PC + 4 + offset × 4 (beq $s0,$s1,L1; at 80012 with offset 2 → 80024). Immediate: the operand is the 16-bit constant in the instruction, sign-extended (addi $s3,$s3,4). Base: the memory address is a register plus the sign-extended 16-bit constant (lw $t0,32($s3) → address = $s3 + 32).

(b) The table, the sketch and the numeric examples above.

Lesson 17 — Synchronization and Pseudoinstructions

Slides 56–57, 59 (§2.11–2.12)

1. Synchronization (sl. 56)

When two processors share memory (P1 writes, then P2 reads) there is a **data race** if they do not synchronise: the result depends on the order of accesses. Hardware support is required: an **atomic read/write** memory operation — no other access to the location is allowed between the read and the write. It can be one instruction (atomic swap register ↔ memory) or an atomic pair.

2. ll and sc (sl. 57) ★

- **Load linked**: `ll rt, offset(rs)` — loads and remembers the address.
- **Store conditional**: `sc rt, offset(rs)` — succeeds only if the location has not changed since the ll: stores rt and returns **1** in rt; otherwise fails, stores nothing and returns **0** in rt.

```
Atomic swap of $s4 with Mem[$s1] (test-and-set a lock):
try: add $t0, $zero, $s4   # copy exchange value
     ll  $t1, 0($s1)       # load linked
     sc  $t0, 0($s1)       # store conditional
     beq $t0, $zero, try   # branch if store fails
     add $s4, $zero, $t1   # put the loaded value in $s4
```

3. Pseudoinstructions (sl. 59) ★

Most assembler instructions map one-to-one to machine instructions. **Pseudoinstructions** are 'figments of the assembler's imagination': the assembler translates them into one or more real instructions, using register **$at** (register 1, assembler temporary).

```
move $t0, $t1        →  add $t0, $zero, $t1
blt  $t0, $t1, L     →  slt $at, $t0, $t1 ;  bne $at, $zero, L
li   $t0, 0x12345678 →  lui $at, 0x1234   ;  ori $t0, $at, 0x5678
bge / bgt / ble      →  slt + beq / bne (Lesson 12)
```

Practice — Lesson 17

(a) Actual — 2022-23 (CSE 305) 6(b) = 2016-17 6(b) = 2014-15 6(b). (b) Actual — 2015-16 6(c). (c) Actual — 2013-14 7(a). (d) Actual — 2019-20 3(b). (e) Actual — 2015-16 5(c).

(a) What support does the MIPS ISA provide for synchronization? Explain with an example. (5–10)

(b) What is a data race? Which MIPS instructions address it and how do they work? (8)

(c) Describe the two synchronization instructions. Write MIPS that does A[0] = X atomically (base of A in $s1, X in $s2). (8)

(d) With an example, describe how pseudoinstructions work. For a performance-critical section, is it always a good idea to write the assembly yourself? (12)

(e) What are pseudoinstructions? Give two examples. (4)

✓ Solution

(a)–(b) Data race: §1. MIPS provides the pair ll/sc (§2): ll reads a location; sc writes it only if no other processor wrote it in between, and reports success (1) or failure (0) in its register. A loop that retries until sc succeeds gives an atomic read-modify-write, e.g. the atomic swap above used to acquire a lock.

(c)

```
try: add $t0, $zero, $s2   # t0 = X (sc overwrites its register, so copy)
     ll  $t1, 0($s1)       # load linked A[0]  (value not needed)
     sc  $t0, 0($s1)       # try to store X into A[0]
     beq $t0, $zero, try   # retry if another processor wrote A[0] in between
```

(d) Example: `blt $t0, $t1, L` is not a MIPS instruction; the assembler emits `slt $at, $t0, $t1` and `bne $at, $zero, L` (also move, li). Writing assembly yourself is **not always a good idea** (fallacy, sl. 66): modern compilers are better at scheduling for modern processors than most programmers, and more lines of hand-written code mean more errors and lower productivity. Profile first; hand-code only a tiny hot spot if the compiler output is measurably worse.

(e) §3: instructions accepted by the assembler but not implemented in hardware; examples move and blt (also li, bgt, bge).

Lesson 18 — Translating and Starting a Program; RISC vs CISC

Slides 58–69 (§2.12, 2.19–2.20) + textbook comparison

1. Translation Hierarchy (sl. 58)

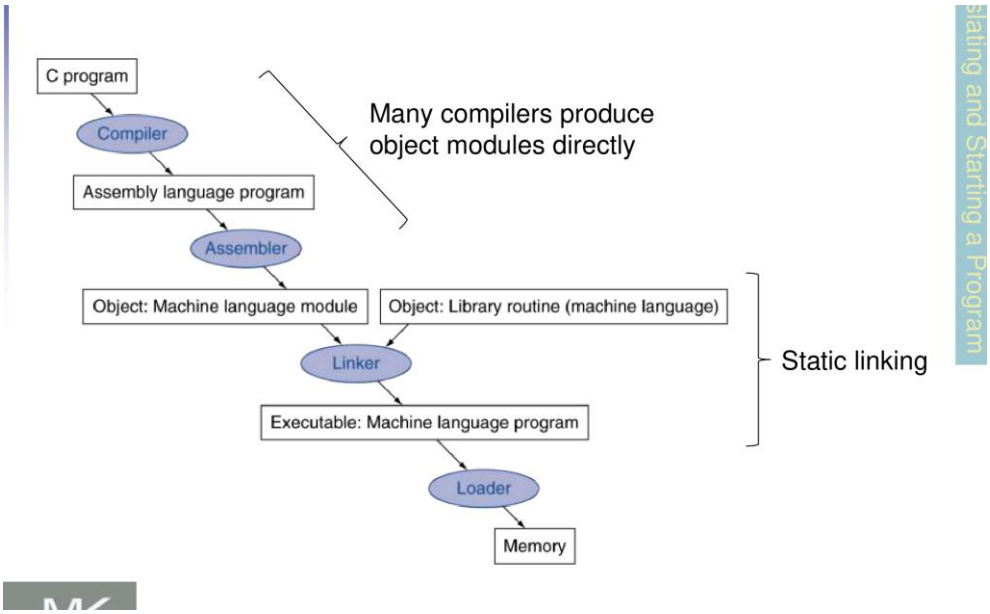


Figure 18.1 — C program → compiler → assembler → linker → loader → memory (sl. 58)

2. Object Module (sl. 60)

Part	Contents
Header	Size and position of the other pieces
Text segment	Translated machine instructions
Static data segment	Data allocated for the life of the program
Relocation info	Instructions/data that depend on absolute addresses
Symbol table	Global definitions and external references (undefined labels)
Debug info	Links machine code to source lines

3. Linker (sl. 61) and Loader (sl. 62)

Linker: (1) merges segments, (2) resolves labels (determines their addresses), (3) patches location-dependent and external references → executable. (Location dependencies could be left for a relocating loader, but with virtual memory every program is loaded at the same virtual addresses.)

Loader: (1) read the header for segment sizes, (2) create the virtual address space, (3) copy text and initialised data into memory (or set page-table entries so they fault in), (4) set up arguments on the stack, (5) initialise registers ($sp, $fp, $gp), (6) jump to the start-up routine, which copies arguments to $a0… and calls main; when main returns it does the exit syscall.

4. Dynamic Linking and Lazy Linkage (sl. 63–64) ★

- **Static linking**: every library routine that may be called is copied into the executable at link time.
- **Dynamic linking (DLL)**: a library routine is linked/loaded only when it is called. The code must be relocatable. Advantages: avoids image bloat from statically linking every (transitively) referenced

library; automatically picks up new library versions; libraries are shared in memory by many programs.

- **Lazy linkage**: the first call goes through an indirection table to a stub that loads the routine ID and jumps to the dynamic linker/loader; the linker finds the routine, maps it, and changes the indirect address so that later calls jump straight to the DLL routine.

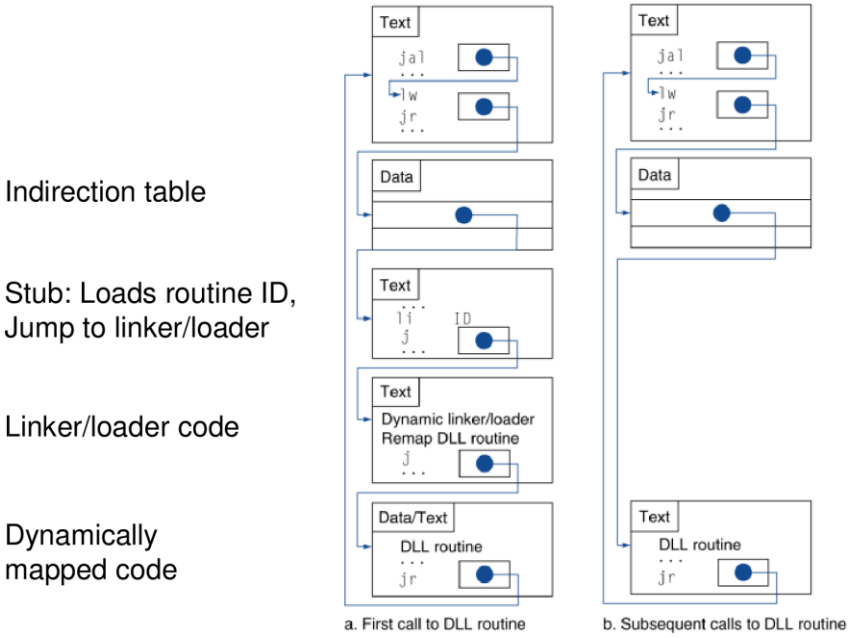


Figure 18.2 — Lazy linkage: (a) first call goes through the stub and dynamic linker; (b) later calls jump directly (sl. 64)

5. Starting Java Applications (sl. 65)

Java program → compiler → class files (Java **bytecodes**, a simple portable instruction set) → the **Java Virtual Machine** interprets the bytecodes (with the Java library routines). A **Just-In-Time (JIT) compiler** compiles the bytecodes of 'hot' methods into native code for the host machine while the program runs.

> **EXAM-READY ANSWER** JIT (2022-23 12(a), 2019-20 1(c)): interpreting bytecodes is portable but slow. The JIT finds the methods that run often, compiles them once into native machine code and caches it; later calls run at native speed, so the program keeps Java's portability while approaching compiled-code performance. The compilation cost is paid only for hot code.

6. Fallacies, Pitfalls and Concluding Remarks (sl. 66–69)

- Fallacy: powerful instruction ⇒ higher performance. Fewer instructions are needed, but complex instructions are hard to implement and may slow down all instructions; compilers make fast code from simple instructions.
- Fallacy: use assembly code for high performance — modern compilers handle modern processors better; more lines ⇒ more errors, less productivity.
- Fallacy: backward compatibility ⇒ the instruction set does not change — it accretes instructions (x86 grew to ~900 instructions, sl. 67).
- Pitfalls: sequential words are not at sequential addresses (increment by 4, not 1); keeping a pointer to an automatic variable after the procedure returns.
- Concluding: the four design principles; layers of software/hardware; MIPS is typical of RISC ISAs, cf. x86.

7. RISC vs CISC, MIPS vs x86 (textbook — asked in older papers)

Feature	RISC (MIPS)	CISC (x86 / IA-32)

Feature	RISC (MIPS)	CISC (x86 / IA-32)
Instructions	Few, simple	Many (hundreds), some very complex (string ops)
Instruction length	Fixed 32 bits, 3 formats	Variable, 1–15 bytes, many formats with prefixes
Operands of arithmetic	Registers only — load/store architecture	Register or memory operand
Number of operands	3 (dest, src1, src2)	2 (one source is also the destination)
Registers	32 general purpose	8 GPRs in IA-32 (16 in x86-64)
Addressing modes	Few (5)	Many (base + scaled index + displacement, segments)
Condition testing	Compare in registers (slt, beq/bne)	Condition codes set as side effects
CPI / control	≈ 1, hardwired, easy to pipeline	Multi-cycle, microcoded, hard to pipeline (translated to micro-ops)
Code size	Larger	Smaller (denser)

Three examples of IA-32 complexity (2011-12 2(e)): variable-length encoding with prefixes and postbytes; string instructions with REP prefixes (one instruction moves a whole block); complex addressing (segment + base + index × scale + displacement) and accreted extensions (MMX, SSE…) — the instruction count keeps growing (sl. 67).

8. Worked Linking Example ★ (2022-23 CSE 305 5(c))

Procedure A: text 0x150, data 0x40; instructions `lw $a0, 0($gp)` (needs X) and `jal 0` (needs B). Procedure B: text 0x350, data 0x50; `sw $a1, 0($gp)` (needs Y) and `jal 0` (needs A). Memory layout (sl. 45): text starts at 0x0040 0000, static data at 0x1000 0000, $gp = 0x1000 8000.

Executable file header		
Text size	0x150 + 0x350 = 0x4A0	
Data size	0x40 + 0x50 = 0x90	

Address	Instruction	How it was patched
0x0040 0000	lw $a0, 0x8000($gp)	X = 0x1000 0000 = $gp − 0x8000 → offset −32768 = 0x8000
0x0040 0004	jal 0x0040 0150	B's text follows A's: 0x0040 0000 + 0x150; field = 0x100054
…	…	
0x0040 0150	sw $a1, 0x8040($gp)	Y = 0x1000 0000 + 0x40 = 0x1000 0040 = $gp − 0x7FC0 → 0x8040
0x0040 0154	jal 0x0040 0000	A's address; field = 0x100000
0x1000 0000	(X)	A's data
0x1000 0040	(Y)	B's data

Practice — Lesson 18

(a) Actual — 2022-23 (CSE 305) 7(d) / 2022-23 12(b) / 2019-20 1(c). (b) Actual — 2013-14 6(a) second part. (c) Actual — 2016-17 4(c) = 2014-15 1(c). (d) Actual — 2020-21 13(d).

(a) Main difference between static and dynamic linking; advantages of dynamic linking; with diagrams, how are DLLs called? (15/10/8)

(b) What are the steps for starting a Java application? (part of 8)

(c) Compare the characteristics of typical CISC and RISC architectures. (6/8)

(d) Main differences between x86 and MIPS? (5)

✓ Solution

(a) Static: all referenced library code is copied into the executable at link time — big image, old library versions stay frozen. Dynamic: library routines are linked/loaded when called — §4 bullets for the advantages; draw Fig. 18.2 and explain the stub → dynamic linker → patched indirection (lazy linkage).

(b) §5: compile to bytecodes (class files) → JVM loads and interprets them with the Java library → JIT compiles hot methods to native code.

(c)–(d) Table in §7 (pick 6–8 rows; for x86 vs MIPS use the MIPS/x86 column wording).

Quick Notes — MIPS ISA

Revision points for Lessons 10–18.

L10 · Operands and principles

- 32 × 32-bit registers; byte-addressed, aligned, big-endian memory; A[i] offset = 4i; spill rarely used variables.
- DP1 simplicity favours regularity (3 operands, fixed 32 bits) · DP2 smaller is faster (32 regs) · DP3 common case fast (immediates, PC-relative) · DP4 good compromises (3 formats, 16-bit fields).

L11 · Encoding

- R: op 6 | rs 5 | rt 5 | rd 5 | shamt 5 | funct 6 · I: op | rs | rt | imm 16 · J: op | addr 26.
- add $t0,$s1,$s2 = 0x02324020. $t0–$t7 = 8–15, $s0–$s7 = 16–23, $a0 = 4, $v0 = 2, $sp 29, $ra 31.
- 2's complement range -2^{31} … $2^{31} - 1$; negate = invert + 1; sign-extend replicates bit 15/7.

L12 · Decisions

- NOT = nor rd, rs, $zero. if: branch on the opposite condition. do-while: test at the bottom.
- No blt/bge: slower compare would slow the clock for all; use slt + beq/bne ($at).

L13 · Procedures

- Args $a0–$a3, results $v0–$v1, $ra, $sp; $s preserved, $t not. jal saves PC + 4 in $ra; jr $ra returns.
- Recursion: base case first → frame ($ra + needed values) → jal → reload → combine → pop → jr.
- Jump table: lw entry k×4, jr.

L14 · Bytes and constants

- lb/lh sign-extend, lbu/lhu zero-extend, sb/sh store rightmost bits (no 'shu' needed).
- 32-bit constant: lui upper; ori lower (not addi). 131075 = lui 2, ori 3.

L15 · Branch/jump addressing

- Branch: target = PC + 4 + off × 4 (±32K words). Jump: (PC + 4)[31:28] : addr × 4 (same 256 MB).
- Far branch: beq → bne over j. Anything else: lui/ori + jr. Unaligned targets impossible.

L16 · Addressing modes

- Immediate, register, base, PC-relative, pseudo-direct — draw the boxes and the calculation.

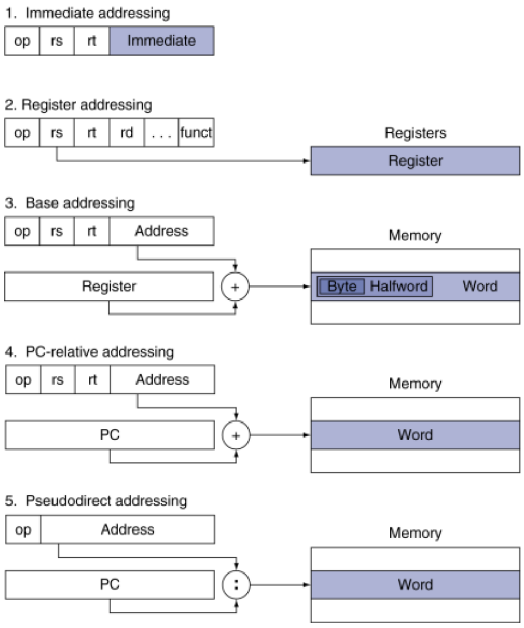
L17 · Sync and pseudo

- Data race; ll/sc (sc returns 1/0); atomic swap loop. Pseudo: move, blt, li via $at.

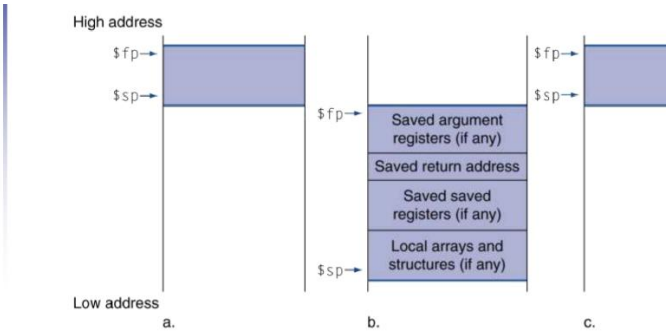
L18 · Translation

- Object module: header, text, data, relocation, symbol table, debug. Linker: merge, resolve, patch. Loader: 6 steps.
- DLL: link on first call via stub/indirection (lazy). JIT compiles hot bytecode methods.
- RISC: fixed length, load/store, 3 operands, 32 regs. CISC/x86: variable length, memory operands, 2 operands, many modes, condition codes.

Key Diagrams to Reproduce



Addressing mode summary (sl. 55) — same as Fig. 16.1



Stack frame (sl. 44) — same as Fig. 13.1